

Hands on 2

Hibernate XML Config implementation walk through Cognizant DN 5.0

Hands-on 2 – Hibernate XML Configuration Implementation Walkthrough

Aim

To study Hibernate XML configuration and understand how Hibernate performs CRUD operations using SessionFactory, Session, Transactions, and XML mapping.

Objective

- Understand Hibernate XML configuration.
- Learn object-relational mapping using XML.
- Understand SessionFactory, Session, and Transaction.
- Learn CRUD operations using Hibernate.

1. Hibernate XML Configuration

Hibernate uses two XML files:

1. hibernate.cfg.xml

This file contains database configuration.

Example:

```
<hibernate-configuration>

  <session-factory>

    <property name="connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>

    <property name="connection.url">
      jdbc:mysql://localhost:3306/ormlearn
    </property>

    <property name="connection.username">
      root
    </property>

    <property name="connection.password">
      root
    </property>

  </session-factory>

</hibernate-configuration>
```

```
<mapping resource="Country.hbm.xml"/>

</session-factory>

</hibernate-configuration>
```

Purpose

- Database connection
- Driver
- Username
- Password
- Dialect
- Mapping files

2. Mapping File (Country.hbm.xml)

Maps Java class with database table.

Example

```
<class name="Country" table="country">
  <id name="code" column="code"/>
  <property name="name" column="name"/>
</class>
```

Purpose

- Maps Java class
- Maps fields
- Maps table
- Maps primary key

2. Object Relational Mapping using XML

Suppose we have

Java Object

Country

code = IN

name = India

↓

Hibernate Mapping



Database Table

country

code name

IN India

Hibernate automatically converts

Java Object



SQL



Database Table

This process is called **Object Relational Mapping (ORM)**.

3. SessionFactory

Definition

SessionFactory is responsible for creating Session objects.

It is created only once during application startup.

Example

```
SessionFactory factory =  
new Configuration()  
.configure()  
.buildSessionFactory();
```

Responsibilities

- Reads hibernate.cfg.xml
- Creates Sessions
- Holds database configuration
- Thread-safe
- Heavyweight object

4. Session

Definition

A Session represents one connection between Hibernate and the database.

Example

```
Session session =  
factory.openSession();
```

Responsibilities

- Executes SQL
- Saves objects
- Updates objects
- Deletes objects
- Fetches objects

Lightweight object.

5. Transaction

A transaction ensures that all database operations execute successfully.

If any operation fails,
all changes are rolled back.

Example

```
Transaction tx =  
session.beginTransaction();
```

6. beginTransaction()

Starts a transaction.

Example

```
Transaction tx =  
session.beginTransaction();
```

Purpose

- Starts database transaction
- Enables commit or rollback

7. commit()

Permanently saves changes.

Example

```
tx.commit();
```

Purpose

- Saves data

- Ends transaction

8. rollback()

Cancels all changes.

Example

```
tx.rollback();
```

Used when

- Exception occurs
- Database error
- Invalid data

9. session.save()

Used to insert a new record.

Example

```
Country c =  
new Country();  
c.setCode("IN");  
c.setName("India");  
session.save(c);
```

Generated SQL

```
INSERT INTO country  
VALUES('IN','India');
```

10. session.createQuery().list()

Retrieves all records.

Example

```
List<Country> countries =  
session.createQuery(  
"FROM Country",  
Country.class  
)list();
```

Generated SQL

```
SELECT *
```

FROM country;

Returns

List<Country>

11. session.get()

Retrieves a single record using Primary Key.

Example

```
Country c =
```

```
session.get(
```

```
Country.class,
```

```
"IN"
```

```
);
```

Generated SQL

```
SELECT *
```

```
FROM country
```

```
WHERE code='IN';
```

12. session.delete()

Deletes a record.

Example

```
Country c =
```

```
session.get(
```

```
Country.class,
```

```
"IN"
```

```
);
```

```
session.delete(c);
```

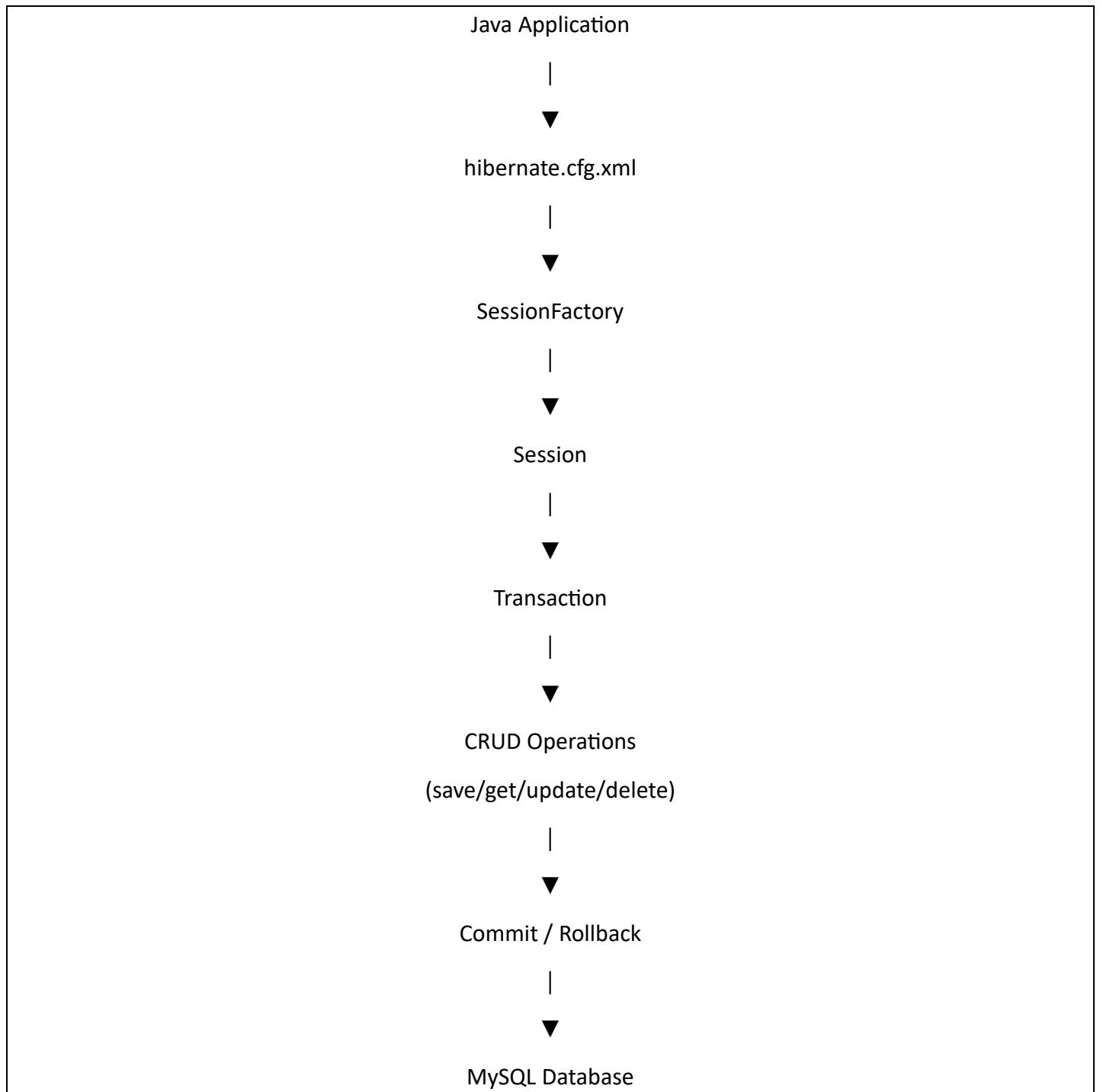
Generated SQL

```
DELETE
```

```
FROM country
```

```
WHERE code='IN';
```

Complete Hibernate Flow



Advantages of Hibernate XML Configuration

- No SQL writing
- Database independent
- Easy object mapping
- Automatic transaction management
- Reusable configuration
- Simplifies CRUD operations
- Supports caching
- Reduces boilerplate code

Difference between XML Mapping and Annotation Mapping

XML Mapping	Annotation Mapping
Mapping stored in .hbm.xml	Mapping stored in Java class
Separate configuration file	Configuration inside entity class
Easy to modify without changing Java code	Easier to write and maintain
Used in older Hibernate projects	Preferred in modern Spring Boot projects